



AULA 1

Introdução às redes neurais e *deep learning*

O PROFESSOR

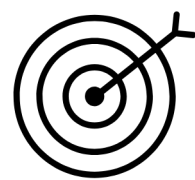
Augusto Baffa

Coordenador de graduação do curso de engenharia da computação e professor do quadro complementar do departamento de informática da PUC-Rio, Augusto é pesquisador chefe em inteligência artificial e jogos no laboratório ICAD/VisionLab e colaborador em pesquisas dos laboratórios Galgos e Exacta. É doutor em informática pela PUC-Rio e mestre em informática pela Unirio, tem MBA executivo em finanças pela IBMEC e MBA em *marketing* pela ESPM e é bacharel em ciências da computação.

O CONVIDADO

Hélio Lopes

Coordenador da Pós-graduação em Ciência de Dados e Analytics, atua como professor do Quadro Principal do Departamento de Informática da PUC-Rio, orientando pesquisas de mestrado e doutorado na área de Ciência de Dados. Sou doutor em Matemática, mestre em Informática e Engenheiro da Computação, todos pela PUC-Rio. Tenho mais de 30 anos de experiência coordenando projetos de pesquisa e desenvolvimento junto a diversas empresas como BNDES, Globo, Ipiranga, Petrobras, Shell, Vale e Americanas, tendo registrado diversas patentes e acumulado diversas premiações nesse contexto.



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Compreender os conceitos básicos de redes neurais *multilayer perceptrons* (MLPs) e o algoritmo de aprendizado *backpropagation*.
- Conhecer os principais conceitos e algoritmos de *deep learning*.
- Entender o que são redes neurais convolucionais (CNNs) e quais são suas aplicações.

Que história é essa?

O objetivo desta disciplina é **introduzir os conceitos de análise descritiva e análise preditiva** para a geração de modelos adequados ao problema em questão. Você verá **como realizar a análise preditiva**, utilizando as principais arquiteturas de redes neurais para isso, e quando deve **analisar a dimensionalidade** em busca de uma melhor *performance* dos modelos.

Também vai aprender a **aplicar os algoritmos de clusterização** para realizar a análise descritiva dos dados, sempre através de exemplos práticos, utilizando a linguagem Python. Foram preparados alguns materiais exclusivos para apoiá-lo nesta jornada ao longo desta disciplina, para que você experimente na prática os tópicos aqui abordados.

Nesta primeira aula, você mergulhará no mundo das redes neurais artificiais, entenderá como surgiram e como funcionam. Deverá compreender quais são os principais desafios em aplicá-las no dia a dia e entender em quais contextos devem ser utilizadas.

Também conhecerá os tão falados algoritmos de *deep learning* e suas profundas redes neurais (DNN, do inglês *deep neural networks*). Ao final, vai entender como funcionam as redes neurais convolucionais (CNN, do inglês *convolutional neural networks*) e suas principais aplicações.

Primeiro, vamos conhecer nossas “redes neurais”

Chamamos de “redes neurais” os grupos de células do nosso cérebro que se comunicam umas com as outras, formando, assim, uma rede de comunicação ou processamento.

O cérebro tem um grande número de **unidades simples** (neurônios) para realizar suas decisões. Esses neurônios ligam e desligam em alguns milissegundos, enquanto os transistores do nosso computador fazem o mesmo em nanossegundos.

Isso quer dizer que o cérebro é “mais lento” que um computador? Sim, entretanto o cérebro realiza tarefas cognitivas complexas (visão, reconhecimento de voz) em décimos de segundo, pois tem uma arquitetura com um **paralelismo massivo**.



Saiba mais

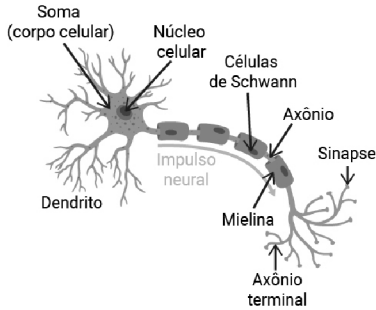
Observe um neurônio natural:

O sinal é recebido de outros neurônios através dos **dentritos**. Esses sinais são chamados de **sinapses**.

Em seguida, os sinais são acumulados no corpo do neurônio, e, quando a soma dos sinais passa de certo limiar (~ 50 mV), um sinal é propagado no **axônio**.

Este é um modelo geral de neurônio, mas existem diversas células nervosas análogas, cada qual com as suas devidas características.

Anatomia do neurônio



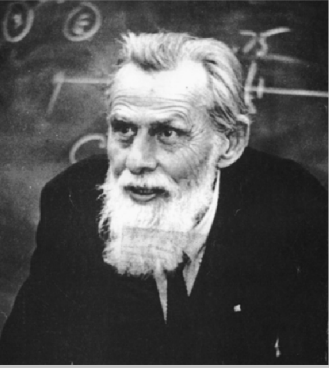
As **sinapses** têm um peso, que pode ser:

- **Excitatório**: quando adicionam um valor positivo à soma dos sinais de outro neurônio.
- **Inibitório**: quando enviam um valor negativo (decrementam) aos sinais enviados ao outro neurônio.

Mas o que são as redes neurais artificiais?

As redes neurais artificiais são uma estrutura matemática baseada na conexão de vários neurônios artificiais que trabalham juntos para responder a um estímulo.

Os neurônios artificiais foram um primeiro conceito introduzido por McCulloch e Pitts em 1943, visando a criar um modelo matemático análogo ao cérebro humano.



Warren Sturgis McCulloch
(16/11/1898 - 24/9/1969)

Neurofisiologista, realizou muitas pesquisas em cibernética e teorias cerebrais.

Walter Pitts
(23/4/1923 - 14/5/1969)

Formado em lógica, trabalhava como cientista cognitivo.



Mas como funciona o neurônio artificial?

O neurônio artificial é um modelo matemático proposto inicialmente por McCulloch e Pitts baseado na regressão logística. Ele procura representar matematicamente como um neurônio utiliza o nível elétrico (ou numérico) que recebe e dispara o sinal em sua saída (que pode ser positivo ou negativo). Agora, vamos entender como ele funciona.

Imagine um problema que pode ser descrito por 11 características e uma 12ª informação que será a resposta desejada para as características mencionadas. Dessa forma, temos um *dataset* com 12 colunas, e ao menos uma será a resposta ou o resultado que o neurônio deve aprender durante o processo de treinamento. Vamos chamar de “atributos” as 11 colunas que definem as características e de “classe” a coluna que contém a resposta.

Sendo um pouco mais formal, podemos dizer que a entrada de um neurônio é um vetor de valores reais $x \in \mathbb{R}^n$, em que n é o número de atributos, e a saída é um $y \in \mathbb{R}$, que será calculada através de um produto escalar. Em sua versão mais simples, a equação do neurônio artificial pode ser representada como a equação linear:

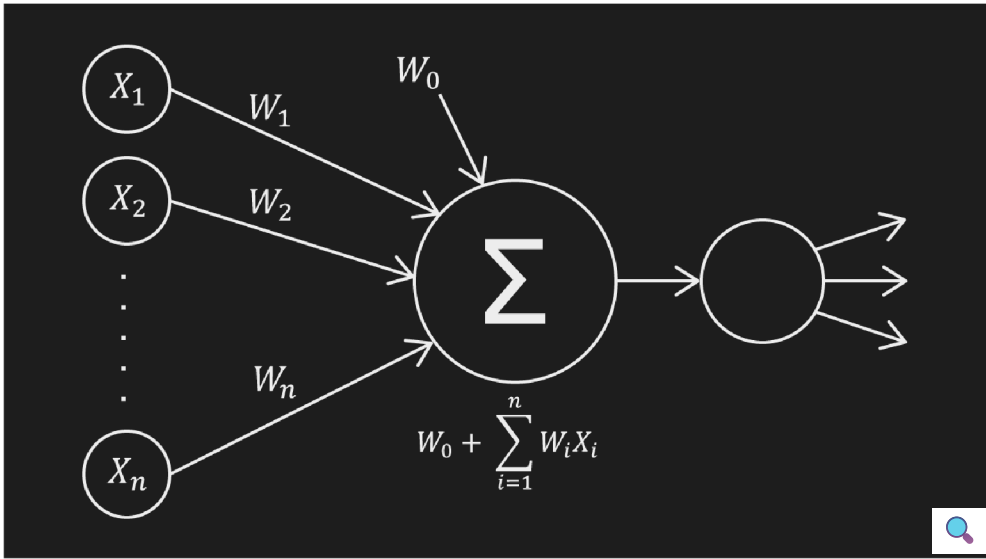
$$z = \overline{w}^T \cdot \overline{x} + b \text{ onde } \overline{w} = (w_1, w_2, \dots, w_n)^T$$

... em que o vetor $w \in \mathbb{R}^n$ é chamado de vetor de pesos sinápticos e o termo escalar $b \in \mathbb{R}$ é uma constante chamada de *bias*. É muito comum considerar o *bias* igual a 1 e adicioná-lo ao vetor de entradas para calcular seus pesos correspondentes.

Conforme vimos anteriormente, as sinapses são responsáveis por processar a informação recebida pelo neurônio. Isso significa dizer que tudo aquilo que o neurônio aprende é guardado no vetor de pesos sinápticos que será usado para processar as sinapses. Na figura a seguir, podemos ver que, ao recebermos a informação representada pelo vetor $\overline{x}$, em cada unidade a informação é ponderada pelos pesos sinápticos e acumulada no corpo celular antes de ser disparada.

$$\bar{x}^* = (x_1, x_2, ..., x_n, 1)$$

Observe, a seguir, o modelo de um neurônio artificial:



Mas como são calculados os pesos sinápticos? Isso ocorre durante o treinamento, quando procuramos associar os melhores pesos dada a resposta esperada e as características da entrada. Esse processo é baseado na neurociência pelas teorias do aprendizado associativo.

Aprendizado associativo

O aprendizado associativo é o **processo que ocorre em nosso cérebro** quando percebermos uma **similaridade ou continuidade entre objetos ou ideias** – por exemplo, quando percebemos que ideias estão relacionadas e assumimos a consequência lógica entre esses fatos. Imagine que você more perto de uma fábrica e toda manhã é acordado pela sirene do primeiro turno. Não só haverá uma associação por experiência de que a sirene toca no mesmo horário e que isso se deve ao fato de que os funcionários estão chegando como também uma conclusão de que provavelmente não será um som agradável após tê-lo acordado diversas vezes.

Redes de *perceptrons*

O *perceptron* é uma rede neural com uma única camada de **neurônios lineares de entrada**, seguida por uma unidade de saída baseada na função *sign*(•):

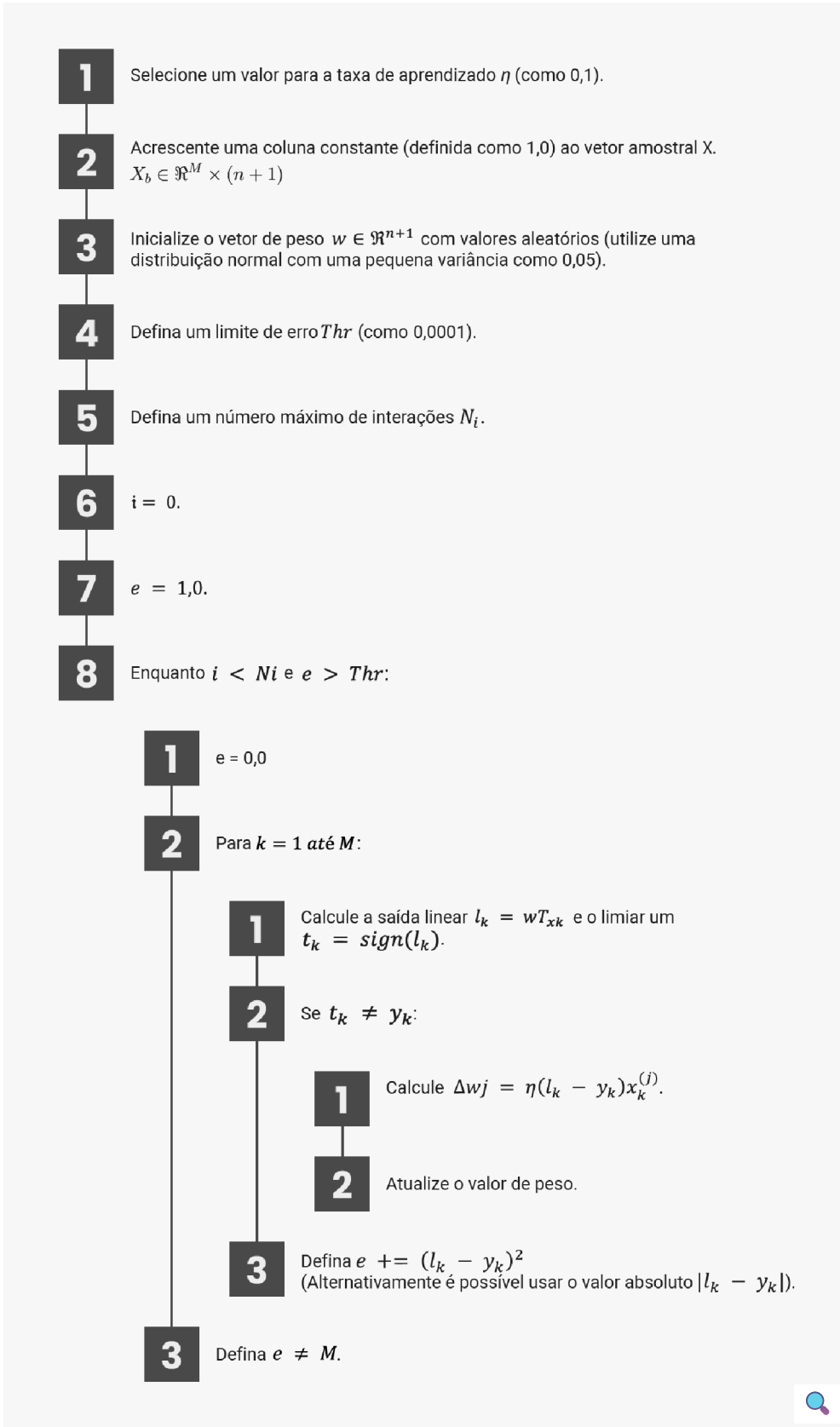
$$y_i = sign(\overline{w}^T \cdot \overline{x} + b) \text{ onde } \overline{w}, \overline{x} \in \mathbb{R}^n$$
$$y_i = \{0, 1\} \text{ ou } y_i = \{-1, 1\}$$

Foi batizado em 1957 por Frank Rosenblatt, que deu nome à sua interpretação de modelo de neurônio artificial.

Rosenblatt (11 de julho de 1928 - 11 de julho de 1971) foi um psicólogo que trabalhou em projetos do Cornell Aeronautical Laboratory, onde conduziu os primeiros trabalhos sobre *perceptrons*, o que resultou em um computador chamado *Mark I Perceptron*, em 1960.

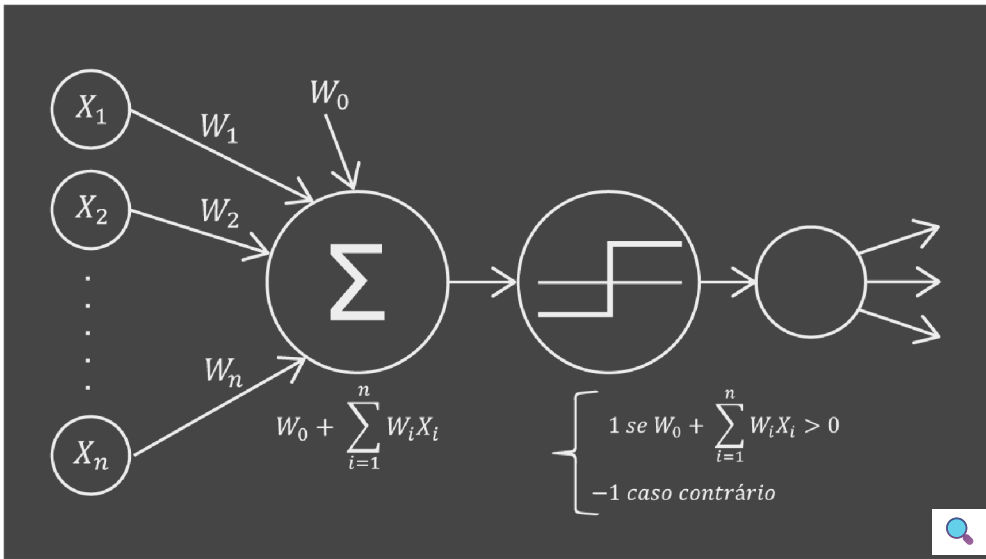
Esse aparelho era capaz de aprender novas habilidades por tentativa e erro, usando um tipo de rede neural que simula os processos de pensamento humano.

O aprendizado do *perceptron* é realizado como apresentado a seguir:



O *perceptron* foi um avanço ao neurônio artificial e adicionou uma função de ativação em degrau para processar o sinal do neurônio e disparar o sinal de saída.

Veja, na figura a seguir, que, após o somatório realizado pelo corpo celular, aplicamos o resultado à função de ativação. A função de ativação é responsável por decidir se o valor computado pelo neurônio é suficiente para dispará-lo ou não.



Infelizmente, os primeiros experimentos demonstraram que os *perceptrons* não eram tão poderosos quanto se esperava.



Reflita

Alguns pesquisadores foram críticos aos primeiros modelos. Por quê?

Se você olhar bem de perto para o modelo de um *perceptron*, ele não vai ser diferente de uma regressão linear múltipla. Mesmo adicionando uma função de ativação, não é adequado para todos os tipos de problemas.

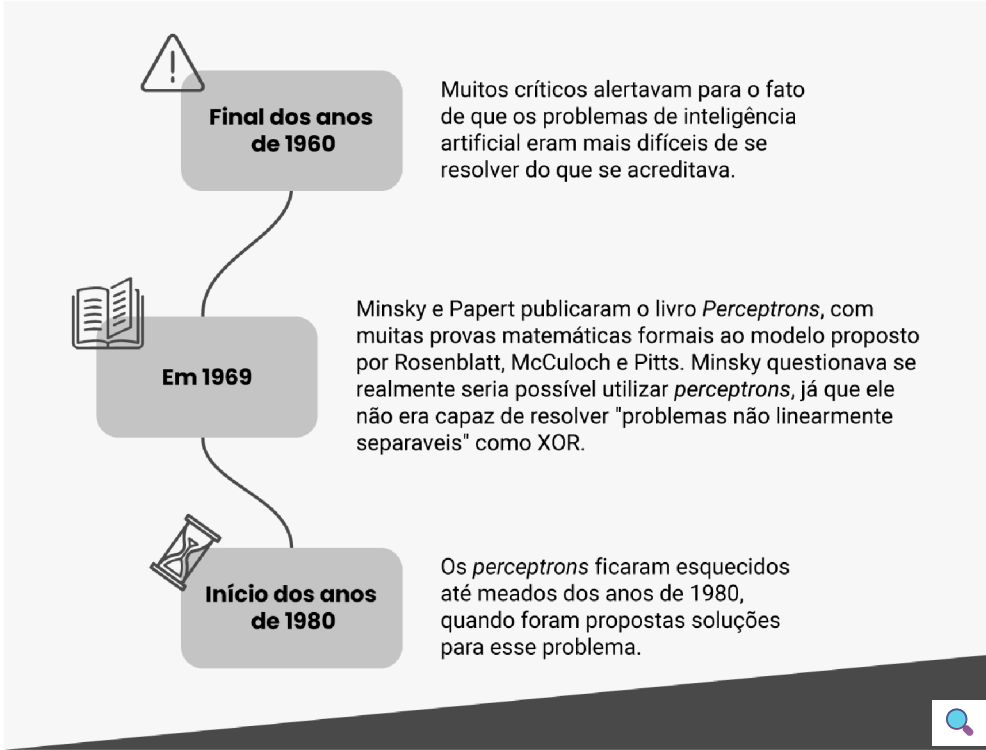


Analise

Quais são os problemas encontrados nos *perceptrons*? Quais tipos de problemas não são resolvidos com esse método?

Os *perceptrons* foram uma boa ideia e um excelente ponto de partida para a criação de modelos de inteligência artificial que se baseiam na biologia e na cognição. Nesse ponto, eles resolvem problemas lineares com muita facilidade, mas os principais problemas reais que gostaríamos de resolver são problemas não lineares.

Problemas não lineares

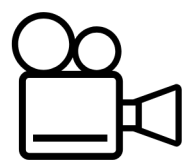


Resolvendo problemas não lineares: *multilayer perceptron*

Com a chegada da microcomputação ao final dos anos de 1970, muitos mais estudantes de ciências da computação puderam dedicar mais e mais horas programando e resolvendo problemas. Logo, algumas atenções também foram voltadas aos *perceptrons* e suas questões em aberto. Não demorou muito para se perceber que, se criarmos uma rede com ao menos uma camada não linear entre entrada e saída, isso levaria a uma combinação altamente não linear, parametrizada e com um número maior de variáveis. A arquitetura

resultante foi chamada de *multilayer perceptron* (MLP), e a camada intermediária entre as camadas de entrada e saída recebeu o nome de “camada oculta”. Por definição, as MLP são redes que contêm uma (ou mais) camada(s) oculta(s).

Nesse ponto, várias arquiteturas começaram a ser propostas para as redes neurais. Assim, podemos dizer que a arquitetura inicial (e, por consequência, mais simples) é a “rede de alimentação direta”, que chamamos pelo nome em inglês *feed-forward networks*.

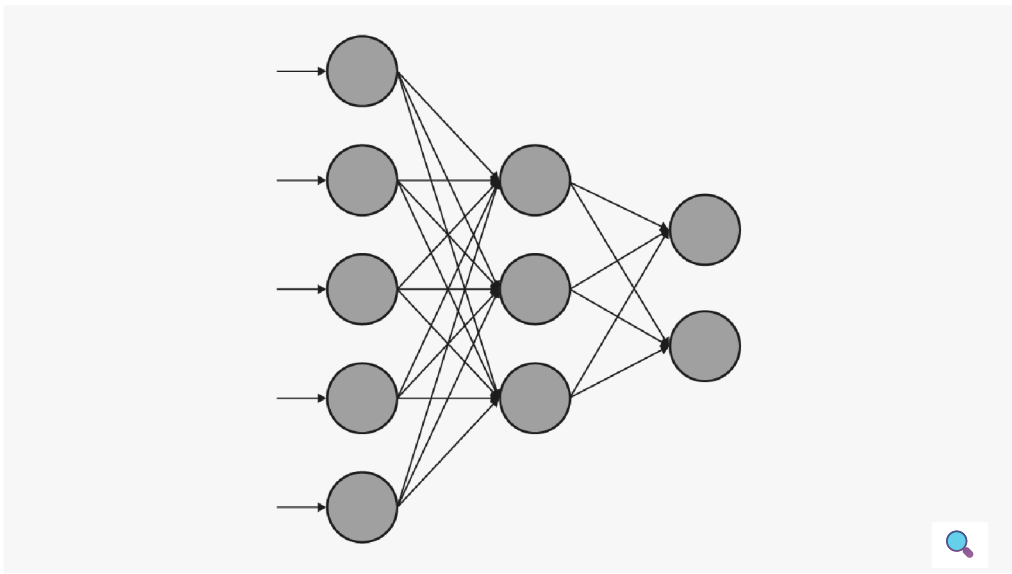


Redes neurais

Neste vídeo você vai retomar conhecimentos básicos sobre conceitos neurais e redes de multicamadas, além disso, as redes serão apresentadas através de exemplos.



As redes neurais *feed-forward*



A **rede *feed-forward*** recebe esse nome para diferenciar entre outras arquiteturas. É o tipo de rede neural mais comum, em que o sinal percorre a rede em uma única direção da camada de entrada até a camada de saída.

As camadas são hiperconectadas, ou seja, cada neurônio de uma camada é conectado a todos os neurônios da camada seguinte. Também define-se que ao menos uma camada oculta utiliza uma função de ativação não linear para manter a não linearidade do modelo.

A **função de ativação de cada camada é fixa**, mas podem-se criar combinações em situações específicas. Já a **ativação de saída é escolhida para atender ao dado que se deseja explicar**, por exemplo: multiclasse, regressão, reconstrução de imagem, etc.

Conheça agora as principais funções de ativação que se pode usar nos *perceptrons*.

Funções de ativação

Uma **função de ativação pode ser qualquer função contínua e diferenciável que pode ser empregada na ativação**. Algumas têm propriedades particulares que permitem obter boa precisão e melhorar a velocidade do processo de aprendizagem.

As funções Relu, Sigmoid, Tanh e Softmax são comumente usadas nos modelos mais modernos (estado da arte). Lembre-se de que é importante entender as propriedades de uma função de ativação para fazer a escolha mais razoável.

Clique nos cards para conhecer mais sobre as funções de ativação.

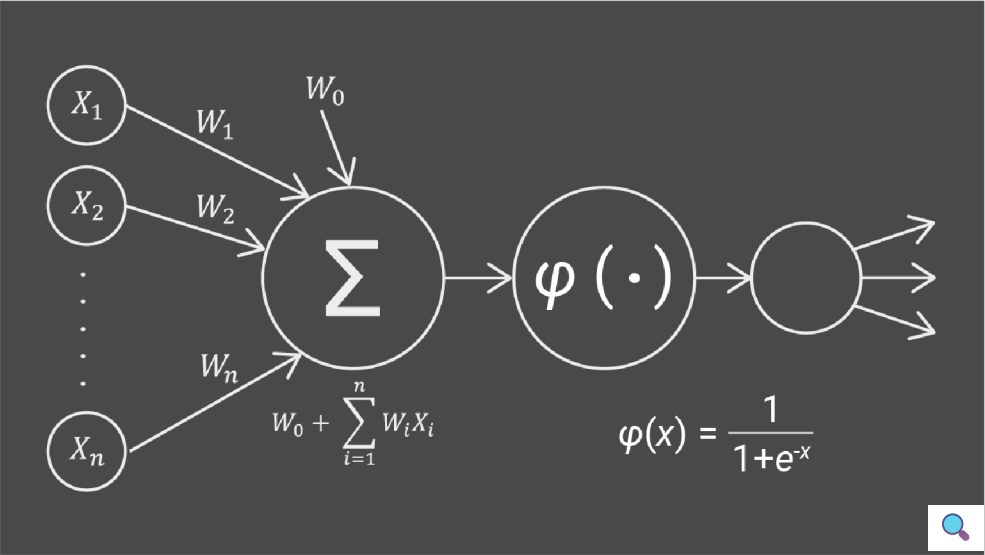
Interativo

Interativo com três caixas brancas em um fundo vermelho. A primeira caixa tem: Função linear: theta abre parênteses x fecha parênteses igual a ax mais b abre parênteses função linear fecha parênteses. Ao clicar nessa, vemos a imagem de um gráfico de linhas com dois eixos: um vertical e um horizontal, marcados respectivamente como theta abre parênteses x fecha parênteses e x. O eixo vertical atravessa o horizontal no meio da imagem, criando uma cruz. Uma linha vermelha diagonal atravessa o gráfico, iniciando abaixo do eixo horizontal e cruzando o eixo vertical acima do ponto zero. Há uma linha pontilhada horizontal (marcada como b) que se estende até o ponto em que a linha vermelha toca o eixo vertical, e um semi círculo pontilhado azul (marcado como a) que vai da parte inferior do eixo horizontal até pouco acima do ponto onde a linha vermelha cruza o eixo vertical. A segunda caixa tem: Função identidade: theta abre parênteses x fecha parênteses igual a x abre parênteses função linear fecha parênteses. Ao clicar nessa, vemos a imagem de um gráfico de linhas com dois eixos: um vertical e um horizontal. O eixo vertical atravessa o horizontal no meio da imagem, criando uma cruz. Os eixos são divididos em intervalos, variando de -4 a +4. No centro do gráfico, onde o eixo horizontal e vertical se cruzam, há uma linha vermelha diagonal que passa de canto a canto. A terceira caixa tem: Limiar (ou degrau): theta abre parênteses v fecha parênteses igual a abre chaves 1 vírgula se v maior que 0 e 0 vírgula se v menor que 0 abre parênteses função limiar ou degrau fecha parênteses. Ao clicar nessa, vemos a imagem de um gráfico com eixos cartesianos, onde o eixo vertical é rotulado como theta abre parênteses v fecha parênteses e o eixo horizontal é rotulado como v. O eixo vertical apresenta uma linha tracejada horizontal à direita, na parte superior, e à esquerda, na parte inferior.

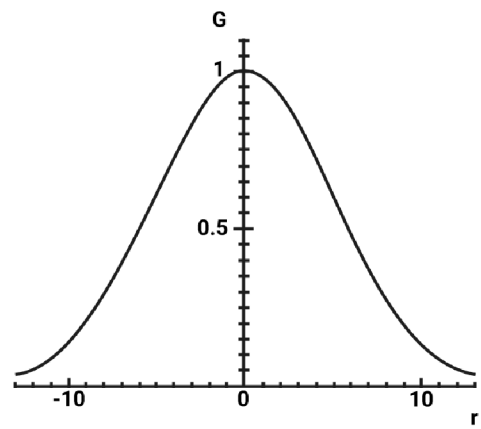
Interativo

Interativo com três caixas brancas em um fundo vermelho. A primeira caixa tem Limiar bipolar theta abre parênteses v fecha parênteses igual a abre chaves 1 vírgula se v maior ou igual a 0 e menos 1 vírgula se v menor que 0 abre parênteses função limiar ou degrau fecha parênteses. Ao clicar nessa, vemos a imagem de um gráfico com eixos cartesianos, onde o eixo vertical é rotulado como theta abre parênteses v fecha parênteses e o eixo horizontal né rotulado como v. O eixo vertical apresenta uma linha contínua e uma tracejada horizontal à direita, na parte superior, e à esquerda, na parte inferior. A segunda caixa tem Sigmoide (logística) theta abre parênteses x fecha parênteses igual a 1 dividido por 1 mais e elevado a menos x abre parênteses função logística fecha parênteses. Ao clicar nessa, vemos a imagem de um gráfico com um fundo branco e uma grade cinza, com o título Sigmoid. Os eixos são rotulados: o eixo horizontal como x e o eixo vertical como y, com uma escala de -10 a 10 no eixo x e de 0 a 1 no eixo y. Há uma curva azul que começa perto da parte inferior esquerda, aumentando gradualmente e se estabilizando perto da parte superior direita do gráfico. A terceira caixa tem Tangente hiperbólica theta abre parênteses x fecha parênteses igual a tanh abre parênteses x fecha parênteses igual a e elevado a x menos e elevado a menos x dividido por e elevado a x mais e elevado a menos x abre parênteses função logística fecha parênteses. Ao clicar nessa, vemos a imagem de um gráfico com um fundo branco e uma grade cinza, com o título Hyperbolic Tangent (tanh), com o eixo horizontal como x e o eixo vertical como tanh abre parênteses x fecha parênteses, com uma escala de -10 a 10 no eixo x e de -1 a 1 no eixo y. A curva azul neste gráfico começa na parte inferior esquerda, passa pelo centro (ponto zero) em uma reta e então sobe até se estabilizar na parte superior direita.

Veja, a seguir, um exemplo de *perceptron* com ativação sigmoide:

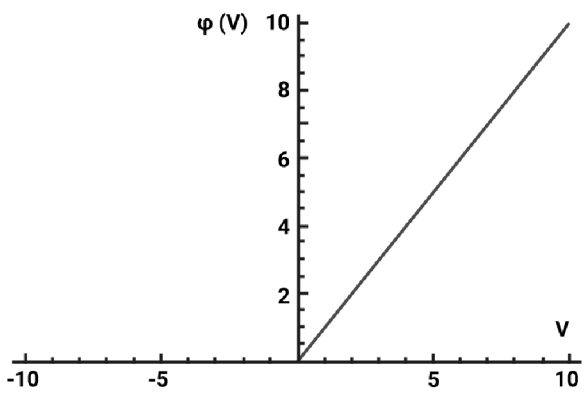


Observe agora a **função gaussiana**, ou função de base radial -
 $\varphi(x) = e^{(-x^2)}$



E a **função retificadora**:

A ReLU é uma família de funções lineares (ou quase lineares para Swish) quando $x > 0$, enquanto diferem quando $x < 0$. Pode-se dizer que alguns neurônios biológicos têm ativação análoga à função ReLU.



Elas também ajudam no aprendizado, pois, em uma rede inicializada aleatoriamente, 50% das unidades ocultas são ativadas, criando uma ativação esparsa e facilitando a convergência do aprendizado.

Interativo

Interativo onde no primeiro slide observamos dois gráficos. À esquerda, vemos um gráfico intitulado Leaky ReLU. Este gráfico tem eixos horizontal e vertical rotulados, respectivamente, como x e y . O eixo x varia de menos 10 a 10, em intervalos de 2 e meio, e o eixo y de 0 a 10, em intervalos de 2. Uma curva azul começa no canto inferior esquerdo, no ponto menos 10 do eixo x e abaixo de 0 no eixo y do gráfico e inclina-se gradualmente para cima à medida que se move para a direita, tornando-se mais íngreme após o ponto 0 do eixo x e do eixo y , chegando ao ponto 10 de ambos os eixos. O segundo gráfico, intitulado ELU, é semelhante ao primeiro, com eixos rotulados x e y . O eixo x varia de menos 10 a 10, em intervalos de 2 e meio, e o eixo y de 0 a 10, em intervalos de 2. Assim como no gráfico anterior, uma curva azul começa no canto inferior esquerdo, no ponto menos 10 do eixo x e abaixo de 0 no eixo y do gráfico e inclina-se gradualmente para cima à medida que se move para a direita, tornando-se mais íngreme após o ponto 0 dos dois eixos, chegando ao ponto 10 de ambos os eixos. A diferença deste para o anterior é que a curva é mais arredondada, enquanto a anterior é mais reta. Abaixo dos gráficos, lê-se: Leaky ReLU θ abre parênteses x fecha parênteses igual a $\max$ abre parênteses 0 vírgula θx fecha parênteses se x é maior ou igual a 1. Na linha de baixo lê-se ELU θ abre parênteses x fecha parênteses igual a $\max$ abre parênteses 0 vírgula θx fecha parênteses se x é maior ou igual a 1. Na linha de baixo lê-se Swish θ abre parênteses x fecha parênteses igual a x sobre $1 + \exp(-\theta x)$ mais e elevado a x fecha parênteses.

A ReLU básica tem um gradiente constante quando $x > 0$, enquanto é nulo para $x < 0$, e é muito empregada em processamento visual.

A Swish é baseada na sigmoide e oferece a vantagem extra de convergir para 0 quando $x \rightarrow 0$, de modo que o efeito não nulo é limitado a uma região curta limitada entre $[-b, 0]$ com $b > 0$. Essa função pode melhorar o desempenho de algumas redes profundas de processamento visual em particular.

A função **Softmax** é usada na camada de saída de quase todas as redes de classificação, pois representa uma distribuição de probabilidade discreta.

Se houver k saídas y_i , o Softmax é calculado como:

$$\varphi(x) = \frac{e^{y_i}}{\sum_{j=1}^k e^{y_i}}$$

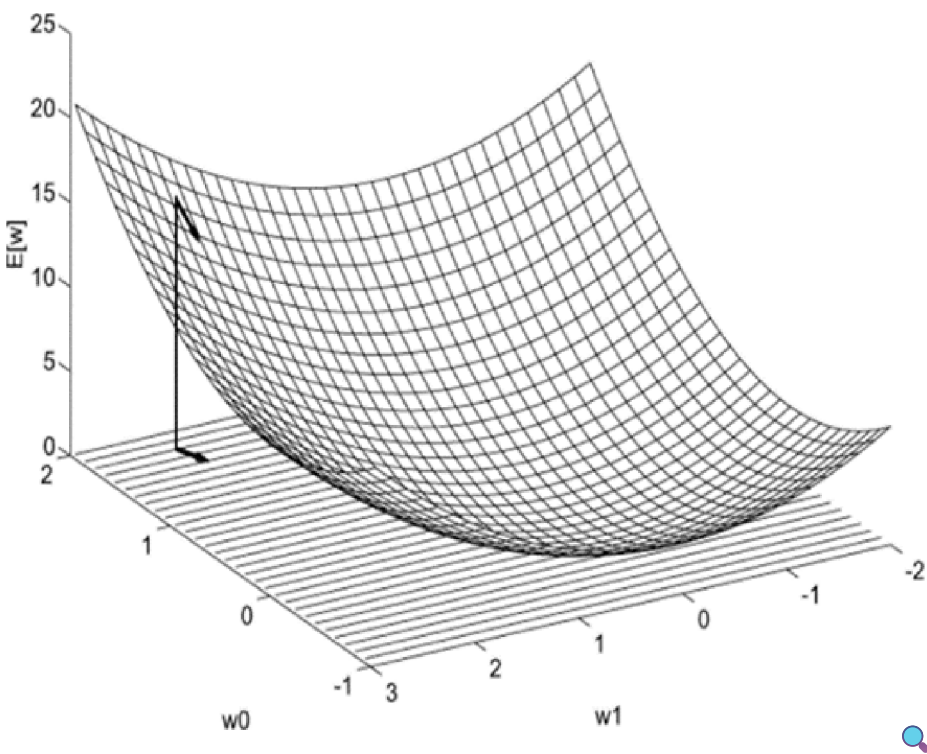


Agora que vimos todas as características de um neurônio, vamos entender como fazemos o treinamento completo de uma rede neural utilizando o aprendizado associativo. Para propagar o erro, corrigindo as sinapses a cada camada, usamos o algoritmo de “retropropagação” dos erros, chamado pelo seu nome em inglês *backpropagation*.

O algoritmo *backpropagation*

O algoritmo *backpropagation* foi proposto em 1986 por David Rumelhart, Geoffrey Hinton e Ronald Williams para **aprender os pesos sinápticos de uma rede multicamadas**.

O algoritmo emprega uma técnica chamada “descida do gradiente” (gradiente descendente) para minimizar o erro quadrático entre a saída da rede e os valores alvos para essas saídas.



Funciona de forma simples: começa com um vetor inicial de pesos arbitrário (selecionado aleatoriamente) e segue modificando os pesos sinápticos repetidamente em pequenos passos definidos pela taxa de aprendizado.

A cada passo, o vetor de pesos é alterado na direção que produz a maior queda do erro total ao longo da superfície da função de erro.

O *backpropagation* não é um algoritmo ótimo, isto é, **não garante sempre a melhor resposta**. Como a rede é inicializada com pesos aleatórios, pode-se executar por várias vezes e encontrar resultados diferentes. Isso ocorre porque o algoritmo de descida do gradiente pode ficar preso em um erro mínimo local.

É possível refazer o treinamento variando os valores iniciais dos pesos para entender melhor a evolução do processo de treinamento para certo conjunto de dados.

Introdução ao aprendizado profundo

Nos últimos anos, ouvimos muito sobre o aprendizado profundo (ou *deep learning*) e sobre como seus resultados são fantásticos. O aprendizado profundo não é algo recente, mas tornou-se viável apenas na última década. Arquiteturas que vamos conhecer em breve (como as redes convolucionais) são estudadas desde o final dos anos de 1980. Ainda, por definição, vamos ver que uma rede neural profunda nada mais é que uma rede MLP com mais camadas ocultas. Então o que de fato torna o aprendizado profundo tão especial? A grande oferta de dados gerados pela internet e o poder computacional que temos à disposição hoje.

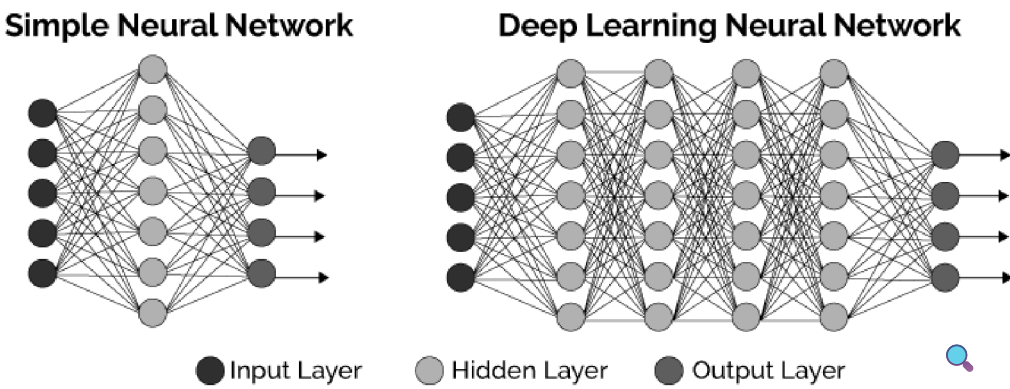


Reflita

Reflita sobre o impacto que as camadas convolucionais e os filtros de *kernel* têm sobre os dados que chegam à rede classificadora/regressora das últimas camadas.

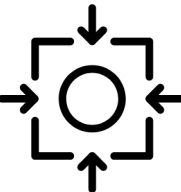
Deep neural networks

As redes neurais profundas são aquelas que têm duas ou mais camadas ocultas na sua arquitetura.



Assim, uma rede MLP clássica pode ser considerada uma rede neural profunda de acordo com sua configuração.

No entanto, as redes neurais clássicas utilizam um conjunto de neurônios horizontais. Dessa forma, como trabalhar com imagens? Imagens são matrizes com largura, altura e canais de cores. Em imagens preto e branco, há apenas uma única matriz bidimensional com valores que definem a iluminação (tons de cinza, entre preto e branco).



Mantendo o foco!

Quando se trata de imagens coloridas, são três canais de cores (ou três matrizes): vermelho, verde e azul. Pensando assim, as DNNs são redes cujas camadas são estruturas tridimensionais (larg. × alt. × canais), diferentemente das MLPs clássicas, que são apenas camadas de uma única dimensão horizontal totalmente conectadas.

Essa arquitetura foi influenciada por estudos neurocientíficos que sugeriram experimentar redes neurais em que as primeiras camadas trabalhassem com estruturas bidimensionais. Dessa forma, a rede neural também poderia analisar a relação espacial das regiões da imagem e, então, extrair uma hierarquia de características estritamente dependentes das suas propriedades geométricas.

De acordo com a neurociência, os humanos não decodificam as imagens diretamente. O processo é sequencial.

Interativo

Aqui, vemos três caixas amarelas, conectadas por setas: passo 1, 2 e 3. Acima, lê-se: Clique em cada um dos passos para conhecê-los. Ao clicar nas caixas, vemos o seguinte texto: Passo 1. Detectam-se elementos de baixo nível (linhas que são orientações). Passo 2. Decompõem-se os elementos em subpropriedades que definem formas cada vez mais complexas, cores diferentes, características estruturais, etc. Passo 3. Ao final, são processadas as características das informações filtradas e define-se a saída.

Isso influenciou os pesquisadores de redes neurais a estudarem técnicas para a decomposição de tais características para que a rede possa pré-processar as informações e selecioná-las com mais facilidade.

Assim, ao final da rede profunda, as últimas camadas podem ser uma rede densa do tipo MLP que será usada para tomar as decisões finais do problema.

Convolutional neural nets

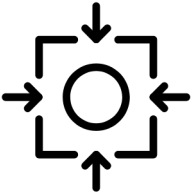
As redes neurais convolucionais (CNN ou *convnet*) são redes **inspiradas no córtex visual que exploram a correlação espacial naturalmente existente em imagens**. Elas são um tipo de rede neural profunda *feedforward* com camadas que não são hiperconectadas.

Fazem o aprendizado através de campos receptivos locais que são processados e resultam em mapas de recursos (também conhecidos como mapas de ativação). Muitas vezes também têm pesos compartilhados entre sinapses, o que auxilia no aprendizado.



Refleta

Refleta sobre o impacto que as camadas convolucionais e os filtros de *kernel* têm sobre os dados que chegam à rede classificadora/regressora das últimas camadas.



Mantendo o foco!

As CNNs são baseadas em camadas que aplicam a operação de convolução. As camadas convolucionais são normalmente aplicadas a entradas bidimensionais (matrizes que representam imagens).

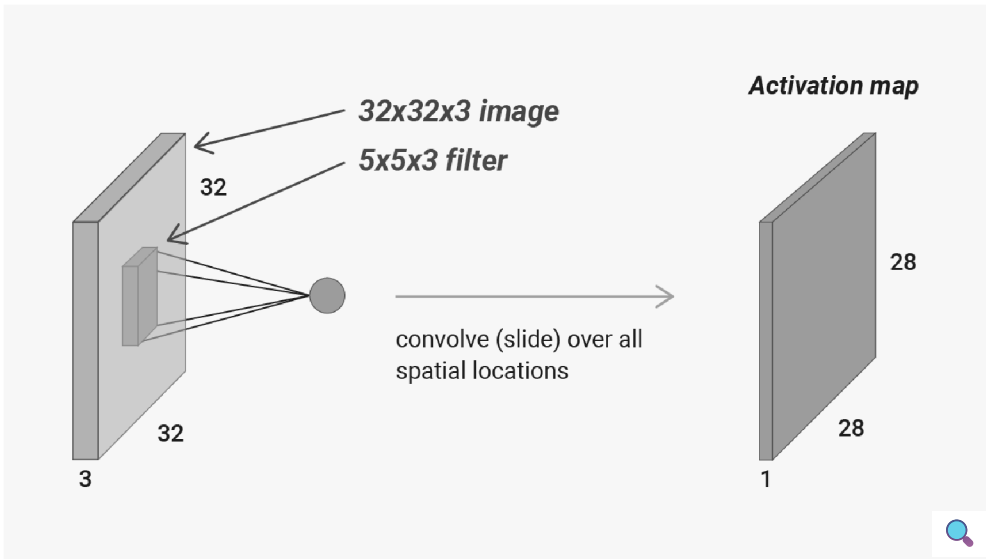
A camada convolucional aprende a filtrar os dados através de uma convolução discreta de um pequeno *kernel* *k* com entrada bidimensional.

Convolução

A convolução é uma operação de somatório do produto entre duas matrizes de tamanhos diferentes que ocorre ao longo da região em que elas se sobrepõem e em razão do deslocamento existente entre elas. É uma operação muito usada para filtros de imagens em computação gráfica (sim, aqueles do Instagram por exemplo), mas podem ser aplicada para tratar qualquer tipo de informação. Por definição, vamos chamar os filtros de *kernels* daqui para a frente.

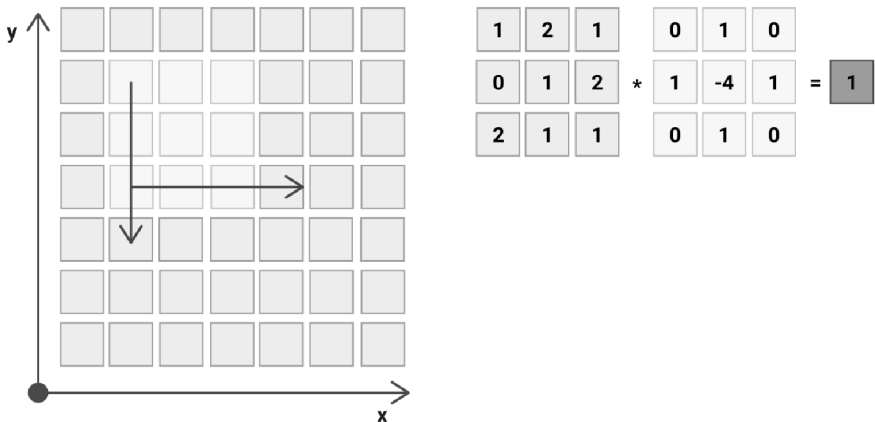
Em uma rede convolucional, uma camada convolucional normalmente é composta por *n kernels* de tamanho fixo, cujos valores são pesos que devem ser otimizados usando o algoritmo de *backpropagation*.

Veja um exemplo na imagem a seguir:



Durante a propagação direta, cada filtro é convolvido através do volume de entrada.

Por exemplo, conforme se observa na imagem a seguir, em uma convolução com *kernel* 3x3, cada conjunto quadrado de 3x3 pixels é filtrado e transformado em um único valor, que corresponde à soma dos pixels superior, inferior, esquerdo e direito.



Fonte: Bonaccorso (201

O *kernel* é deslocado horizontalmente e verticalmente, produzindo a soma da multiplicação elemento a elemento dos elementos correspondentes. Cada operação leva à saída de um único pixel. O exemplo utiliza um *kernel* laplaciano:

$$K = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$



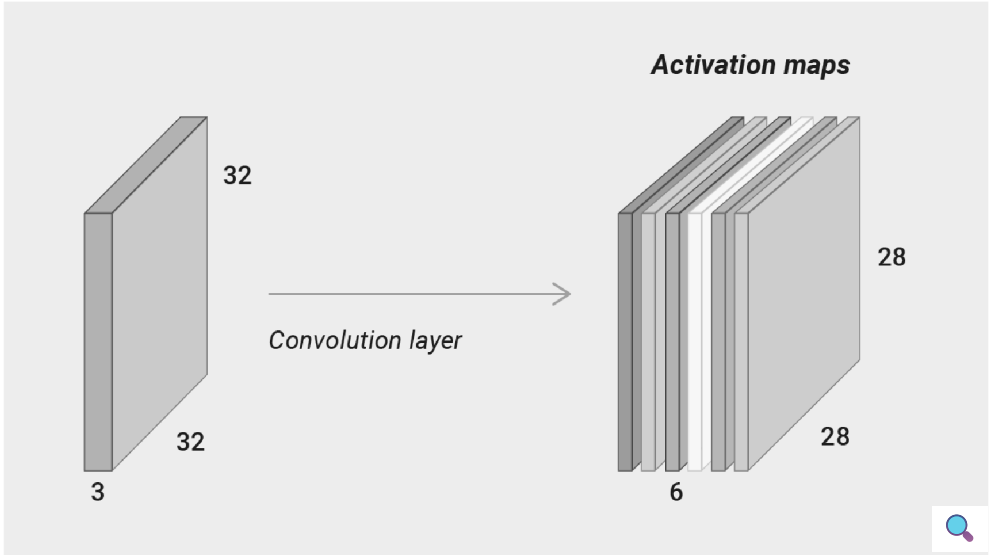
Seja $X \in R^{(w \times h)}$ e $k \in R^{(n \times m)}$, a convolução $X * k$ é definida como

$$(X * k)(x, y) = \sum_{i \in [0, n-1], j \in [0, m-1]} k(i, j) X(x + i, y + j)$$



... onde w é a largura e h é a altura da matriz de entrada, e n é a largura e m é a altura da matriz de *kernel*.

A saída de um CONV é outro volume, uma sequência de mapas de ativação (*feature maps*). O tamanho da sequência de mapas de ativação de um volume é igual à dimensão de profundidade desse volume.



Padding

Quando um *kernel* ($n \times m$ com $n, m > 1$) é deslocado sobre uma imagem e chega ao final de uma dimensão, existem três possibilidades: *valid padding*, *same padding* ou *zero padding*.

Clique nos cards para ver as informações de cada um.

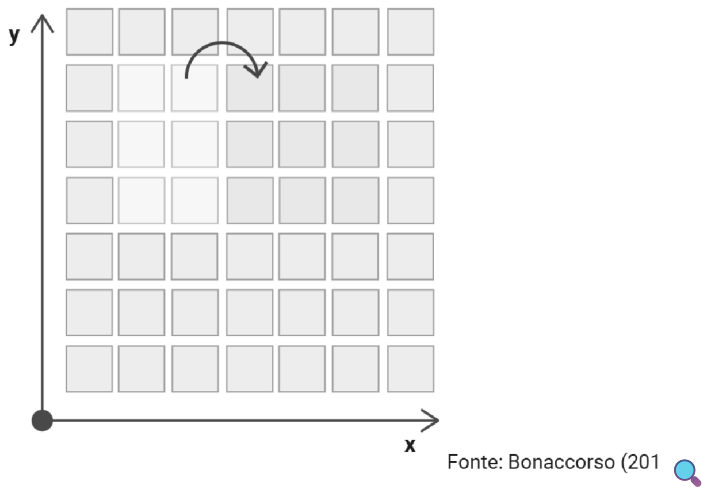
Interativo

No interativo, vemos três caixas brancas em um fundo azul: a primeira diz valid padding, outra same padding, e a terceira zero-padding abre parênteses P fecha parênteses. Ao clicar nas caixas, vemos o seguinte texto: Valid padding: Não continuar mesmo que a imagem resultante seja menor que a original. Se X for uma matriz w vezes h, a saída de convolução resultante terá dimensões iguais a abre parênteses w menos n mais 1 fecha parênteses vezes abre parênteses h menos m mais 1 fecha parênteses. Same padding: Manter as dimensões originais. Adiciona n menos 1 colunas em branco e m menos 1 linhas em branco para permitir que o kernel se desloque sobre a imagem original. Gera um número de pixels igual às dimensões iniciais. Zero-padding abre parênteses P fecha parênteses: Pode ser empregado para aumentar o tamanho de um mapa de características, por exemplo, para alinhá-lo com outro. É usado quando as dimensões de entrada precisam ser preservadas na saída. Abaixo, vê-se a imagem de uma matriz quadrada que contém vários números dentro de células. As células ao longo das bordas superior e inferior da matriz contêm o número zero. No centro, um grupo de células está destacado com um fundo rosa. As células dentro deste grupo contêm os seguintes números, de cima para baixo e da esquerda para a direita: 35, 19, 25, 6; 13, 22, 16, 53; 4, 3, 7, 10; 9, 8, 1, 3. Segue o texto: Cria uma moldura sem informação (n pixels pretos são adicionados antes e depois de cada lado). Os valores zero são adicionados simetricamente a cada fatia de profundidade de um volume de entrada para aumentar suas dimensões.

Stride

É um hiperparâmetro que controla como o filtro é 'deslizado' pelo volume de entrada durante a convolução. O *stride* define o número de pixels a serem ignorados durante cada turno.

Por exemplo, um valor definido para (1, 1) corresponde a uma convolução padrão. Veja o exemplo. Nesse caso, foram definidos passos de (2, 1) – então, o *kernel* avança em dois passos:



Pooling

O *pooling* também é chamado de camada de “subamostragem”. É uma camada não treinável que só serve para **reduzir a complexidade quando o número de convoluções é muito alto**.

Essa operação reduz a informação, transformando cada grupo de pontos de entrada (pixels em uma imagem) em um único valor seguindo uma estratégia definida por sua configuração.

As estratégias que podem ser usadas são:

Interativo

No interativo, acima, lê-se o texto: clique em cada fórmula para saber mais. Abaixo, vê-se duas fórmulas ligadas por chaves. A primeira é $F_{\max}(x_{ij})$ e a segunda é $F_{\text{avg}}(x_{ij})$. Ao clicar na primeira, lê-se: Max Pooling - Cada grupo bidimensional de m vezes n pixels é transformado em um único pixel cujo valor é o maior do grupo. A segunda fórmula é $F_{\text{avg}}(x_{ij}) = \frac{1}{m \cdot n} \sum_{i=1}^m \sum_{j=1}^n x_{ij}$. Ao clicar na segunda, lê-se: Avg Pooling - Cada grupo bidimensional de m vezes n pixels é transformado em um único pixel cujo valor é a média do grupo.

Em alguns casos, a dimensionalidade da matriz original pode ser reduzida com perda de informações.

Exemplo: primeiras camadas, em que a granularidade das características é grosseira. Assim, as camadas de subamostragem fornecem robustez média e aumentam a capacidade de generalização da rede.

21	8	8	12
12	19	9	7
8	10	4	3
18	12	9	10

avg pooling

15	9
12	7

max pooling

21	12
18	10

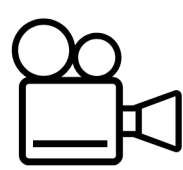
O *pooling* pode ser usado entre duas camadas sucessivas de CONVs para reduzir a complexidade do modelo e atenuar o *overfitting*.

Funciona reduzindo as dimensões espaciais (largura x altura) do volume de entrada e aplicado a cada fatia de profundidade do volume de entrada.

A profundidade do volume de saída é igual à do volume de entrada.

Outras camadas importantes

Interativo



Redes neurais profundas

Neste vídeo você vai conhecer um pouco mais sobre redes neurais profundas, seus modelos e suas aplicações.



Arquiteturas DCN

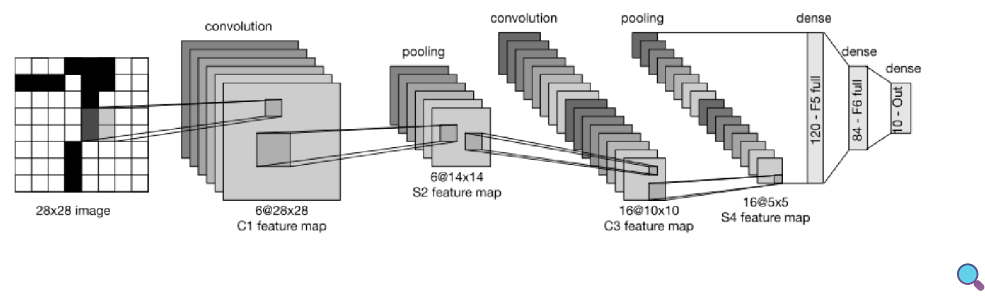
Para finalizar esta aula, conheça algumas arquiteturas convolucionais bastante conhecidas e identifique cada uma das características das redes neurais convolucionais que foram aplicadas nelas.

Lembre-se de que, em uma CNN, encontram-se um ou mais estágios, cada qual composto por camadas de convolução (CONV), de subamostragem (POOL) e de normalização de contraste (*batch normalization*).

Ao final de toda CNN, é comum encontrar uma ou mais camadas completamente conectadas que finalizam a rede utilizando uma camada Softmax (módulo de classificação).

LeNet

A LeNet é uma das primeiras CNNs de que se tem notícia. É baseada no resultado dos estudos de Yann LeCun no final dos anos de 1980. Foi publicada em 1998 e demonstra como as redes neurais podem identificar objetos em uma foto.

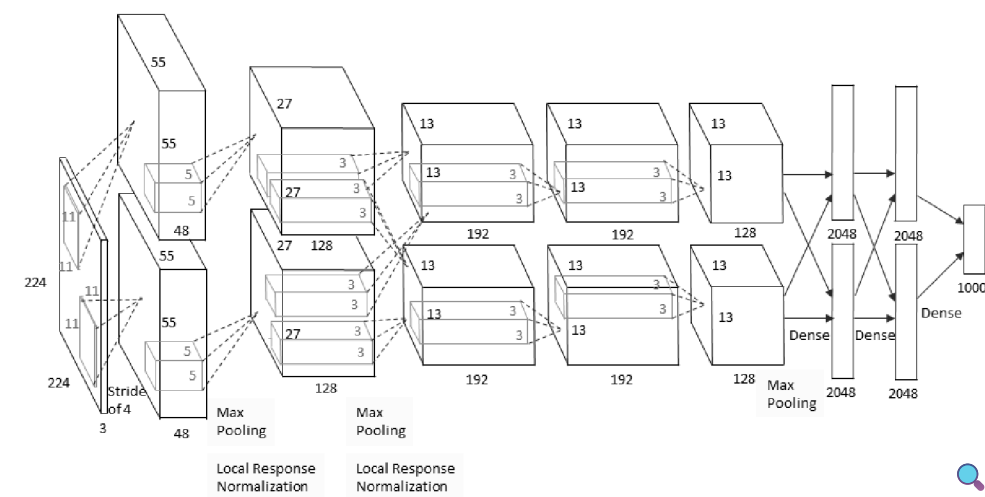


Fonte: Zhang; Lipton; Li; Smola (2023).

Rede AlexNet

A AlexNet é uma rede proposta por Krizhevsky em colaboração com Sutskever e Hinton (este último um dos autores do algoritmo *backpropagation*, lembra?).

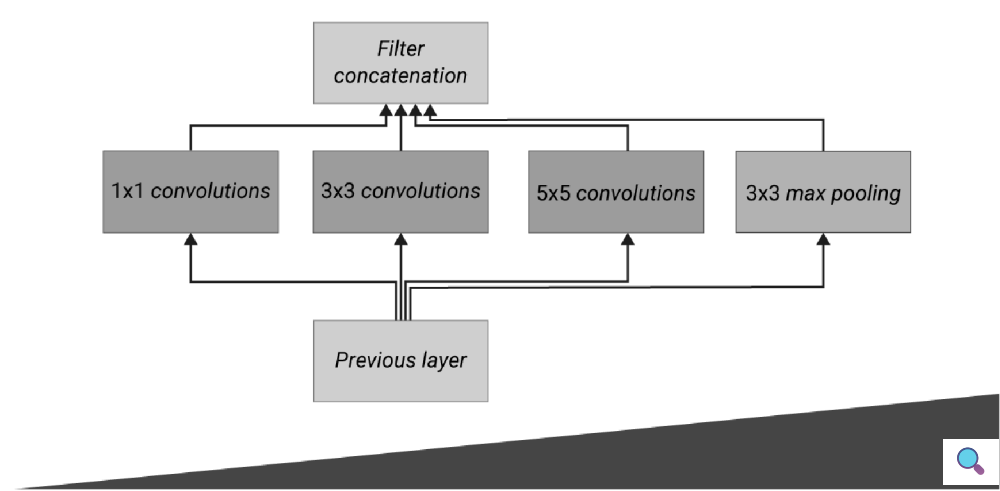
A rede ganhou notoriedade por ter ganhado a edição de 2012 do campeonato chamado ImageNet Large Scale Visual Recognition Challenge, melhorando o recorde da época em pouco mais de 10%. A ImageNet propunha cerca de ~1,2M de imagens de alta resolução para treinamento e ~1.000 imagens por classe (1.000 classes). Até então, soluções incluíam características produzidas manualmente, estudadas e melhoradas por décadas.



Fonte: Krizhevsky, Sutskever e Hilton (2012).

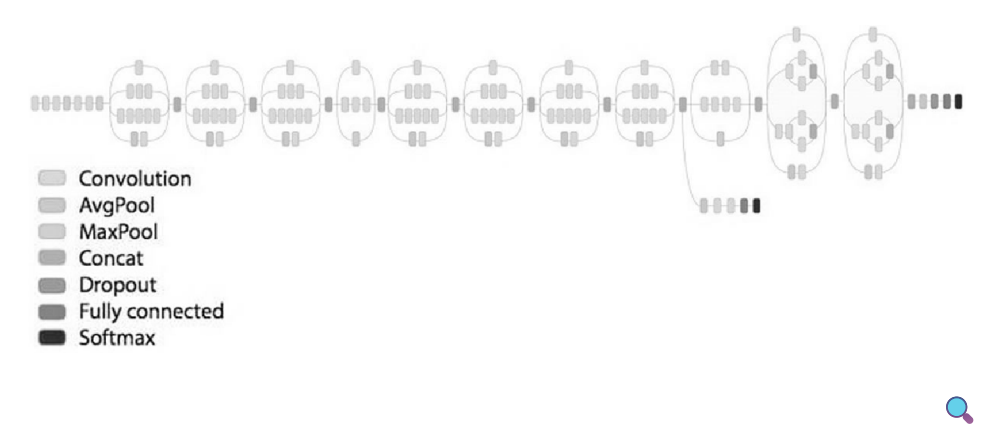
GoogLeNet/Inception, 2014

Em 2014, a Google apresentou uma arquitetura nova na ImageNet chamada “Inception”. A ideia geral era a de que um módulo poderia formar diversas configurações nas quais a rede aprenderia qual deveria ser usada em cada etapa.



Haveria 22 camadas Inception, e a própria rede deveria aprender em quais casos deveria haver mais ou menos camadas, decidindo “passar direto” quando achasse necessário.

Ao final, a rede completa teria aparência semelhante à da figura a seguir:

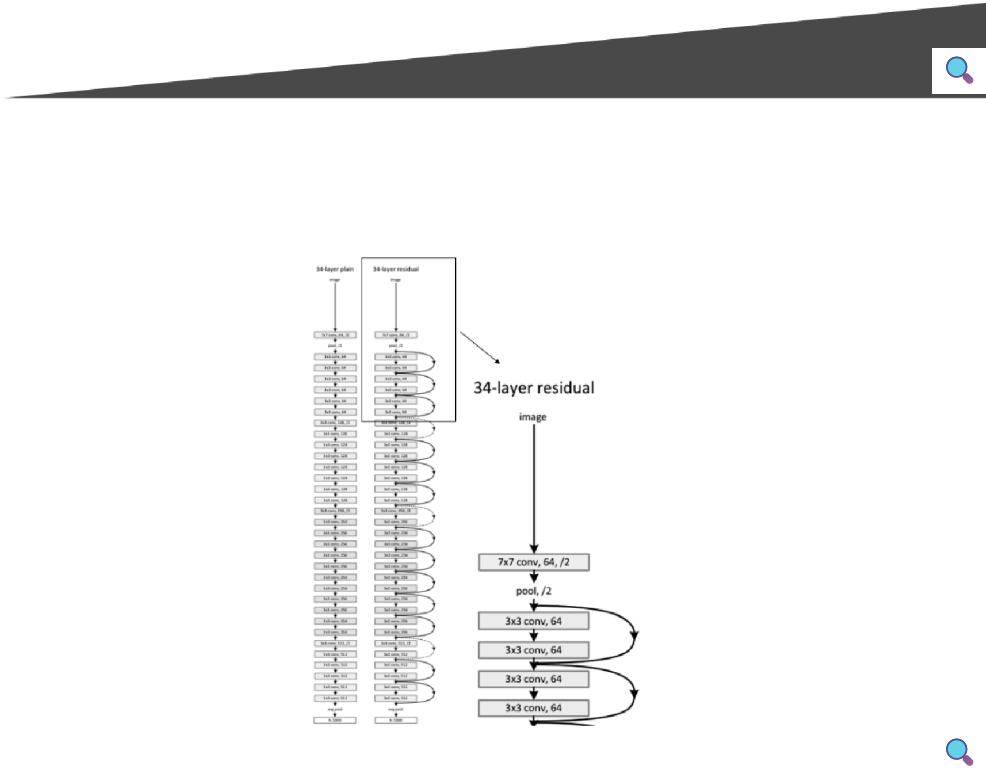
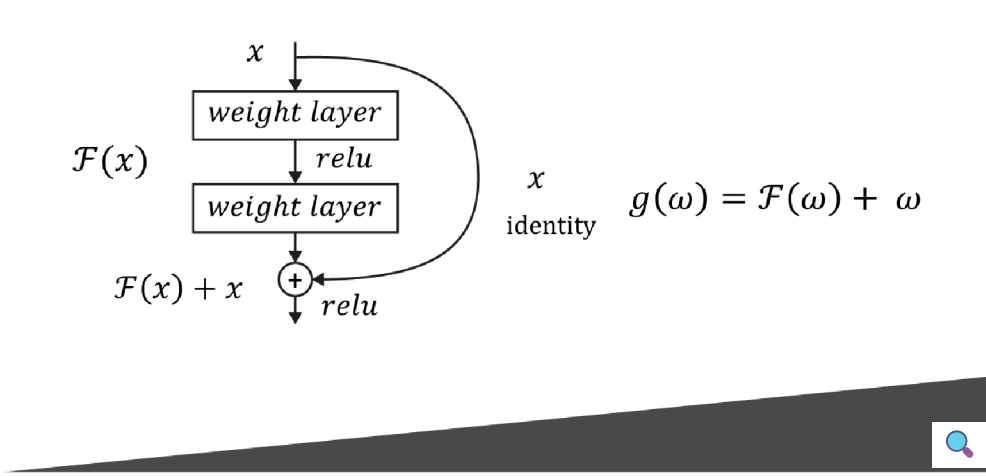


Veja que existe uma saída final e uma saída secundária. Ao todo, haveria diversas saídas antecipadas que a rede poderia utilizar para classificar as diversas imagens que deveriam ser aprendidas.

ResNet, 2015

Para finalizar, conheça a ResNet, uma rede proposta pela Microsoft Research também para o ImageNet.

Nessa arquitetura, a rede deveria também aprender a continuar o trajeto pelas camadas convolucionais ou pular as camadas, encurtando o percurso do sinal analisado. Seguiu uma ideia básica: introduzir “conexões de atalho de identidade” que pulam uma ou mais camadas.



Fonte: He, Zhang, Ren e Sun (2015).

A Microsoft ResNet foi proposta com 152 camadas e atingiu bons resultados nos problemas de classificação, detecção e localização. Para isso, foi treinada utilizando 8 GPUs por 2/3 semanas.

Para validar a hipótese de que quanto mais camadas, melhor a acurácia obtida por uma rede, a Microsoft Research testou a ResNet

por aumentar o número de camadas para até 1.202, mas não conseguiu bons resultados.



Referências

Clique aqui para acessar as referências e créditos desta aula.



APRENDIZADO de máquina e aprendizado profundo e redes neurais convolucionais. **Issuepert**, 9 dez. 2019. Disponível em: <https://issuepert.com/m/87>. Acesso em: 30 mar. 2023.

BONACCORSO, Guiseppe. **Machine learning algorithms**. 2. ed. [s.l.]: Packt Publishing, 2018.

DONALD Hebb. **UBC Archives Photograph Collection**. 2003. Disponível em: <https://open.library.ubc.ca/collections/arphotos/items/1.0148833>. Acesso em: 28 mar. 2023.

GÉRON, Aurelien; **Hands-on machine learning with Scikit-Learn, Keras & TENSORFlow**. 2. ed. [s.l.]: O'Reilly Media, 2019.

GOODFELLOW, Ian; BENGIO, Yoshua; COURVILLE, Aaron.**Deep learning**. [s.l.]: MIT Press; 2016.

HE, Kaiming; ZHANG, Xiangyu; Ren, SHAOQING; SUN, Jian. Deep residual learning for image recognition. **arXiv**, 2015.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HILTON, Geoffrey E. ImageNet classification with deep convolutional neural networks. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 2012, Nevada. **Proceedings** [...]. Nevada, 2012. P. 1-9. Disponível em: https://proceedings.neurips.cc/paper_files/paper/2012/file/c39986Paper.pdf. Acesso em: 28 mar. 2023.

LECUN, Yan; KAVUKCUOGLU, Koray, FARABET, Clément. Convolutional networks and applications in vision In: ISCAS 2010 – 2010 IEEE INTERNATIONAL SYMPOSIUM ON CIRCUITS AND SYSTEMS: NANO-BIO CIRCUIT FABRICS AND SYSTEMS, 2010, Paris. **Proceedings** [...]. Paris, 2010. p. 253-256.

MISNKY, Marvin; PAPERT, Seymour. **Perceptrons**: an introduction to computational geometry. Cambridge: The MIT Press, 1969.

MORENO-DÍAZ, Roberto; MORENO-DÍAZ, Arminda. On the legacy of W.S. McCulloch. **Bio Systems**, v. 88, n. 3, 2007.

SHALEV-SHWARTZ, Shai.; BEN-DAVID, Shai. **Understanding machine learning**: from theory to algorithms. [s.l.]: Cambridge University Press, 2014.

SOBRAL JÚNIOR, Geraldo Alves. **Propriedades dinâmicas de redes de neurônios com estímulos externos e diluição assimétrica**. 2005. Dissertação (Mestrado em Física) – Universidade Federal de Alagoas, Maceió, 2005.

SOARES, Vinicius. “Nobel da Computação” vai para os pais do deep learning. **Instituto de Engenharia**, 2019. Disponível em: <https://www.institutodeengenharia.org.br/site/2019/04/01/nobel-da-computacao%E2%80%8B-vai-para-os-pais-do-deep-learning/>. Acesso em: 28 mar. 2023.

WALTER Pitts in 1954. **Wikipédia**, 2022. Disponível em: https://commons.wikimedia.org/wiki/File:1954_Walter_Pitts_and_a. Acesso em: 28 mar. 2023.

WARREN S. McCulloch, 1898-1969. **Semantic Scholar**, 2007. Disponível em: <https://www.semanticscholar.org/paper/On-the-legacy-of-W.S.-McCulloch-Moreno-D%C3%ADaz-MorenoD%C3%ADaz/8056242a82ecc5e0064d4ff187fb07c5853fe8>. Acesso em: 28 mar. 2023.

Zhang, A; Lipton, Z. C.; Li, M.; Smola, A. J. **LeNet-5 architecture**.

7 dez. 2023. Disponível em:

https://commons.wikimedia.org/wiki/File:LeNet-5_architecture.svg. Acesso em: 10 Jul. 2025.

Banco de imagens Envato, Freepik, Pexels, Pixabay e Shutterstock.